JavaScript is disabled on your browser.

**Java SE 27 & JDK 27**

# Class Calendar

java.lang.Object
    java.util.Calendar

**All Implemented Interfaces:**

`Serializable`, `Cloneable`, `Comparable<Calendar>`

**Direct Known Subclasses:**

`GregorianCalendar`

---

```
public abstract class Calendar
extends Object
implements Serializable, Cloneable, Comparable<Calendar>
```

The `Calendar` class is an abstract class that provides methods for converting between a specific instant in time and a set of `calendar fields` such as `YEAR`, `MONTH`, `DAY_OF_MONTH`, `HOUR`, and so on, and for manipulating the calendar fields, such as getting the date of the next week. An instant in time can be represented by a millisecond value that is an offset from the *Epoch*, January 1, 1970 00:00:00.000 GMT (Gregorian).

The class also provides additional fields and methods for implementing a concrete calendar system outside the package. Those fields and methods are defined as `protected`.

Like other locale-sensitive classes, `Calendar` provides a class method, `getInstance`, for getting a generally useful object of this type. `Calendar`'s `getInstance` method returns a `Calendar` object whose calendar fields have been initialized with the current date and time:

```
Calendar rightNow = Calendar.getInstance();
```

A `Calendar` object can produce all the calendar field values needed to implement the date-time formatting for a particular language and calendar style (for example, Japanese-Gregorian, Japanese-Traditional). `Calendar` defines the range of values returned by certain calendar fields, as well as their meaning. For example, the first month of the calendar system has value `MONTH == JANUARY` for all calendars. Other values are defined by the concrete subclass, such as `ERA`. See individual field documentation and subclass documentation for details.

## Getting and Setting Calendar Field Values

The calendar field values can be set by calling the `set` methods. Any field values set in a `Calendar` will not be interpreted until it needs to calculate its time value (milliseconds from the Epoch) or values of the calendar fields.

Calling the `get`, `getTimeInMillis`, `getTime`, `add` and `roll` involves such calculation. Unless otherwise specified, any `Calendar` method containing the parameter `int field` will throw an `ArrayIndexOutOfBoundsException` if the specified field is out of range (`field < 0 || field >= FIELD_COUNT`).

## Leniency

`Calendar` has two modes for interpreting the calendar fields, *lenient* and *non-lenient*. When a `Calendar` is in lenient mode, it accepts a wider range of calendar field values than it produces. When a `Calendar` recomputes calendar field values for return by `get()`, all of the calendar fields are normalized. For example, a lenient `GregorianCalendar` interprets `MONTH == JANUARY`, `DAY_OF_MONTH == 32` as February 1.

When a `Calendar` is in non-lenient mode, it throws an exception if there is any inconsistency in its calendar fields. For example, a `GregorianCalendar` always produces `DAY_OF_MONTH` values between 1 and the length of the month. A non-lenient `GregorianCalendar` throws an exception upon calculating its time or calendar field values if any out-of-range field value has been set.

## First Week

`Calendar` defines a locale-specific seven day week using two parameters: the first day of the week and the minimal days in first week (from 1 to 7). These numbers are taken from the locale resource data or the locale itself when a `Calendar` is constructed. If the designated locale contains "fw" and/or "rg" Unicode extensions, the first day of the week will be obtained according to those extensions. If both "fw" and "rg" are specified, the value from the "fw" extension supersedes the implicit one from the "rg" extension. They may also be specified explicitly through the methods for setting their values.

When setting or getting the `WEEK_OF_MONTH` or `WEEK_OF_YEAR` fields, `Calendar` must determine the first week of the month or year as a reference point. The first week of a month or year is defined as the earliest seven day period beginning on `getFirstDayOfWeek()` and containing at least `getMinimalDaysInFirstWeek()` days of that month or year. Weeks numbered ..., -1, 0 precede the first week; weeks numbered 2, 3,... follow it. Note that the normalized numbering returned by `get()` may be different. For example, a specific `Calendar` subclass may designate the week before week 1 of a year as week *n* of the previous year.

## Calendar Fields Resolution

When computing a date and time from the calendar fields, there may be insufficient information for the computation (such as only year and month with no day of month), or there may be inconsistent information (such as Tuesday,

July 15, 1996 (Gregorian) -- July 15, 1996 is actually a Monday). `Calendar` will resolve calendar field values to determine the date and time in the following way.

If there is any conflict in calendar field values, `Calendar` gives priorities to calendar fields that have been set more recently. The following are the default combinations of the calendar fields. The most recent combination, as determined by the most recently set single field, will be used.

For the date fields:

```
YEAR + MONTH + DAY_OF_MONTH
YEAR + MONTH + WEEK_OF_MONTH + DAY_OF_WEEK
YEAR + MONTH + DAY_OF_WEEK_IN_MONTH + DAY_OF_WEEK
YEAR + DAY_OF_YEAR
YEAR + DAY_OF_WEEK + WEEK_OF_YEAR
```

For the time of day fields:

```
HOUR_OF_DAY
AM_PM + HOUR
```

If there are any calendar fields whose values haven't been set in the selected field combination, `Calendar` uses their default values. The default value of each field may vary by concrete calendar systems. For example, in `GregorianCalendar`, the default of a field is the same as that of the start of the Epoch: i.e., `YEAR = 1970`, `MONTH = JANUARY`, `DAY_OF_MONTH = 1`, etc.

**Note:** There are certain possible ambiguities in interpretation of certain singular times, which are resolved in the following ways:

1. 23:59 is the last minute of the day and 00:00 is the first minute of the next day. Thus, 23:59 on Dec 31, 1999 < 00:00 on Jan 1, 2000 < 00:01 on Jan 1, 2000.
2. Although historically not precise, midnight also belongs to "am", and noon belongs to "pm", so on the same day, 12:00 am (midnight) < 12:01 am, and 12:00 pm (noon) < 12:01 pm

The date or time format strings are not part of the definition of a calendar, as those must be modifiable or overridable by the user at runtime. Use `DateFormat` to format dates.

## Field Manipulation

The calendar fields can be changed using three methods: `set()`, `add()`, and `roll()`.

**`set(f, value)`** changes calendar field `f` to `value`. In addition, it sets an internal member variable to indicate that calendar field `f` has been changed. Although calendar field `f` is changed immediately, the calendar's time value in milliseconds is not recomputed until the next call to `get()`, `getTime()`, `getTimeInMillis()`, `add()`, or `roll()` is made. Thus, multiple calls to `set()` do not trigger multiple, unnecessary computations. As a result of changing a calendar field using `set()`, other calendar fields may also change, depending on the calendar field, the calendar field value, and the calendar system. In addition, `get(f)` will not necessarily return `value` set by the call to the `set` method after the calendar fields have been recomputed. The specifics are determined by the concrete calendar class.

*Example*: Consider a `GregorianCalendar` originally set to August 31, 1999. Calling `set(Calendar.MONTH, Calendar.SEPTEMBER)` sets the date to September 31, 1999. This is a temporary internal representation that resolves to October 1, 1999 if `getTime()` is then called. However, a call to `set(Calendar.DAY_OF_MONTH, 30)` before the call to `getTime()` sets the date to September 30, 1999, since no recomputation occurs after `set()` itself.

**`add(f, delta)`** adds `delta` to field `f`. This is equivalent to calling `set(f, get(f) + delta)` with two adjustments:

> **Add rule 1**. The value of field `f` after the call minus the value of field `f` before the call is `delta`, modulo any overflow that has occurred in field `f`. Overflow occurs when a field value exceeds its range and, as a result, the next larger field is incremented or decremented and the field value is adjusted back into its range.
>
> **Add rule 2**. If a smaller field is expected to be invariant, but it is impossible for it to be equal to its prior value because of changes in its minimum or maximum after field `f` is changed or other constraints, such as time zone offset changes, then its value is adjusted to be as close as possible to its expected value. A smaller field represents a smaller unit of time. `HOUR` is a smaller field than `DAY_OF_MONTH`. No adjustment is made to smaller fields that are not expected to be invariant. The calendar system determines what fields are expected to be invariant.

In addition, unlike `set()`, `add()` forces an immediate recomputation of the calendar's milliseconds and all fields.

*Example*: Consider a `GregorianCalendar` originally set to August 31, 1999. Calling `add(Calendar.MONTH, 13)` sets the calendar to September 30, 2000. **Add rule 1** sets the `MONTH` field to September, since adding 13 months to August gives September of the next year. Since `DAY_OF_MONTH` cannot be 31 in September in a `GregorianCalendar`, **add rule 2** sets the `DAY_OF_MONTH` to 30, the closest possible value. Although it is a smaller field, `DAY_OF_WEEK` is not

adjusted by rule 2, since it is expected to change when the month changes in a `GregorianCalendar`. In leap years, the adjustment accounts for the leap day in February to ensure the day of month is valid for the year.

**`roll(f, delta)`** adds `delta` to field `f` without changing larger fields. This is equivalent to calling `add(f, delta)` with the following adjustment:

> **Roll rule**. Larger fields are unchanged after the call. A larger field represents a larger unit of time. `DAY_OF_MONTH` is a larger field than `HOUR`.

*Example*: See `GregorianCalendar.roll(int, int)`.

**Usage model**. To motivate the behavior of `add()` and `roll()`, consider a user interface component with increment and decrement buttons for the month, day, and year, and an underlying `GregorianCalendar`. If the interface reads January 31, 1999 and the user presses the month increment button, what should it read? If the underlying implementation uses `set()`, it might read March 3, 1999. A better result would be February 28, 1999. Furthermore, if the user presses the month increment button again, it should read March 31, 1999, not March 28, 1999. By saving the original date and using either `add()` or `roll()`, depending on whether larger fields should be affected, the user interface can behave as most users will intuitively expect.

**Since:**

1.1

**See Also:**

`System.currentTimeMillis()`, `Date`, `GregorianCalendar`, `TimeZone`, `DateFormat`, Serialized Form

## *Nested Class Summary*

**Nested Classes**

| Modifier and Type | Class | Description |
|---|---|---|
| `static class` | **`Calendar.Builder`** | `Calendar.Builder` is used for creating a `Calendar` from various date-time parameters. |

## *Field Summary*

**Fields**

| Modifier and Type | Field | Description |
|---|---|---|

| | | |
|---|---|---|
| static final int | **ALL_STYLES** | A style specifier for `getDisplayNames` indicating names in all styles, such as "January" and "Jan". |
| static final int | **AM** | Value of the `AM_PM` field indicating the period of the day from midnight to just before noon. |
| static final int | **AM_PM** | Field number for `get` and `set` indicating whether the `HOUR` is before or after noon. |
| static final int | **APRIL** | Value of the `MONTH` field indicating the fourth month of the year in the Gregorian and Julian calendars. |
| protected boolean | **areFieldsSet** | True if `fields[]` are in sync with the currently set time. |
| static final int | **AUGUST** | Value of the `MONTH` field indicating the eighth month of the year in the Gregorian and Julian calendars. |
| static final int | **DATE** | Field number for `get` and `set` indicating the day of the month. |
| static final int | **DAY_OF_MONTH** | Field number for `get` and `set` indicating the day of the month. |
| static final int | **DAY_OF_WEEK** | Field number for `get` and `set` indicating the day of the week. |
| static final int | **DAY_OF_WEEK_IN_MONTH** | Field number for `get` and `set` indicating the |

| | | |
|---|---|---|
| | | ordinal number of the day of the week within the current month. |
| static final int | **DAY_OF_YEAR** | Field number for get and set indicating the day number within the current year. |
| static final int | **DECEMBER** | Value of the MONTH field indicating the twelfth month of the year in the Gregorian and Julian calendars. |
| static final int | **DST_OFFSET** | Field number for get and set indicating the daylight saving offset in milliseconds. |
| static final int | **ERA** | Field number for get and set indicating the era, e.g., AD or BC in the Julian calendar. |
| static final int | **FEBRUARY** | Value of the MONTH field indicating the second month of the year in the Gregorian and Julian calendars. |
| static final int | **FIELD_COUNT** | The number of distinct fields recognized by get and set. |
| protected int[] | **fields** | The calendar field values for the currently set time for this calendar. |
| static final int | **FRIDAY** | Value of the DAY_OF_WEEK field indicating Friday. |
| static final int | **HOUR** | Field number for get and set indicating the |

| | | |
|---|---|---|
| | | hour of the morning or afternoon. |
| static final int | **HOUR_OF_DAY** | Field number for `get` and `set` indicating the hour of the day. |
| protected boolean[] | **isSet** | The flags which tell if a specified calendar field for the calendar is set. |
| protected boolean | **isTimeSet** | True if then the value of `time` is valid. |
| static final int | **JANUARY** | Value of the `MONTH` field indicating the first month of the year in the Gregorian and Julian calendars. |
| static final int | **JULY** | Value of the `MONTH` field indicating the seventh month of the year in the Gregorian and Julian calendars. |
| static final int | **JUNE** | Value of the `MONTH` field indicating the sixth month of the year in the Gregorian and Julian calendars. |
| static final int | **LONG** | A style specifier for `getDisplayName` and `getDisplayNames` equivalent to `LONG_FORMAT`. |
| static final int | **LONG_FORMAT** | A style specifier for `getDisplayName` and `getDisplayNames` indicating a long name used for format. |

| | | |
|---|---|---|
| static final int | **LONG_STANDALONE** | A style specifier for `getDisplayName` and `getDisplayNames` indicating a long name used independently, such as a month name as calendar headers. |
| static final int | **MARCH** | Value of the `MONTH` field indicating the third month of the year in the Gregorian and Julian calendars. |
| static final int | **MAY** | Value of the `MONTH` field indicating the fifth month of the year in the Gregorian and Julian calendars. |
| static final int | **MILLISECOND** | Field number for `get` and `set` indicating the millisecond within the second. |
| static final int | **MINUTE** | Field number for `get` and `set` indicating the minute within the hour. |
| static final int | **MONDAY** | Value of the `DAY_OF_WEEK` field indicating Monday. |
| static final int | **MONTH** | Field number for `get` and `set` indicating the month. |
| static final int | **NARROW_FORMAT** | A style specifier for `getDisplayName` and `getDisplayNames` indicating a narrow name used for format. |
| static final int | **NARROW_STANDALONE** | A style specifier for `getDisplayName` and `getDisplayNames` |

| | | |
|---|---|---|
| | | indicating a narrow name independently. |
| static final int | **NOVEMBER** | Value of the MONTH field indicating the eleventh month of the year in the Gregorian and Julian calendars. |
| static final int | **OCTOBER** | Value of the MONTH field indicating the tenth month of the year in the Gregorian and Julian calendars. |
| static final int | **PM** | Value of the AM_PM field indicating the period of the day from noon to just before midnight. |
| static final int | **SATURDAY** | Value of the DAY_OF_WEEK field indicating Saturday. |
| static final int | **SECOND** | Field number for get and set indicating the second within the minute. |
| static final int | **SEPTEMBER** | Value of the MONTH field indicating the ninth month of the year in the Gregorian and Julian calendars. |
| static final int | **SHORT** | A style specifier for getDisplayName and getDisplayNames equivalent to SHORT_FORMAT. |
| static final int | **SHORT_FORMAT** | A style specifier for getDisplayName and getDisplayNames |

| | | |
|---|---|---|
| | | indicating a short name used for format. |
| static final int | **SHORT_STANDALONE** | A style specifier for `getDisplayName` and `getDisplayNames` indicating a short name used independently, such as a month abbreviation as calendar headers. |
| static final int | **SUNDAY** | Value of the `DAY_OF_WEEK` field indicating Sunday. |
| static final int | **THURSDAY** | Value of the `DAY_OF_WEEK` field indicating Thursday. |
| protected long | **time** | The currently set time for this calendar, expressed in milliseconds after January 1, 1970, 0:00:00 GMT. |
| static final int | **TUESDAY** | Value of the `DAY_OF_WEEK` field indicating Tuesday. |
| static final int | **UNDECIMBER** | Value of the `MONTH` field indicating the thirteenth month of the year. |
| static final int | **WEDNESDAY** | Value of the `DAY_OF_WEEK` field indicating Wednesday. |
| static final int | **WEEK_OF_MONTH** | Field number for `get` and `set` indicating the week number within the current month. |
| static final int | **WEEK_OF_YEAR** | Field number for `get` and `set` indicating the |

| | | |
|---|---|---|
| | | week number within the current year. |
| static final int | YEAR | Field number for get and set indicating the year. |
| static final int | ZONE_OFFSET | Field number for get and set indicating the raw offset from GMT in milliseconds. |

## Constructor Summary

**Constructors**

| Modifier | Constructor | Description |
|---|---|---|
| protected | Calendar() | Constructs a Calendar with the default time zone and the default FORMAT locale. |
| protected | Calendar(TimeZone zone, Locale aLocale) | Constructs a calendar with the specified time zone and locale. |

## Method Summary

**All Methods** | **Static Methods**

| Modifier and Type | Method | Description |
|---|---|---|
| abstract void | add(int field, int amount) | Adds or subtracts the specified amount of time to the given calendar field, based on the calendar's rules. |
| boolean | after(Object when) | Returns whether this Calendar represents a time after the time |

| | | |
|---|---|---|
| | | represented by the specified `Object`. |
| `boolean` | **`before`**`(Object when)` | Returns whether this `Calendar` represents a time before the time represented by the specified `Object`. |
| `final void` | **`clear`**`()` | Sets all the calendar field values and the time value (millisecond offset from the Epoch) of this `Calendar` undefined. |
| `final void` | **`clear`**`(int field)` | Sets the given calendar field value and the time value (millisecond offset from the Epoch) of this `Calendar` undefined. |
| `Object` | **`clone`**`()` | Creates and returns a copy of this object. |
| `int` | **`compareTo`**`(Calendar anotherCalenda` | Compares the time values (millisecond offsets from the Epoch) represented by two `Calendar` objects. |
| `protected void` | **`complete`**`()` | Fills in any unset fields in the calendar fields. |
| `protected abstract void` | **`computeFields`**`()` | Converts the current millisecond time value `time` to calendar field values in `fields[]`. |
| `protected abstract void` | **`computeTime`**`()` | Converts the current calendar field values in `fields[]` to the millisecond time value `time`. |

| | | |
|---|---|---|
| boolean | **equals**(Object obj) | Compares this Calendar to the specified Object. |
| int | **get**(int field) | Returns the value of the given calendar field. |
| int | **getActualMaximum**(int field) | Returns the maximum value that the specified calendar field could have, given the time value of this Calendar. |
| int | **getActualMinimum**(int field) | Returns the minimum value that the specified calendar field could have, given the time value of this Calendar. |
| static Set<String> | **getAvailableCalendarType** | Returns an unmodifiable Set containing all calendar types supported by Calendar in the runtime environment. |
| static Locale[] | **getAvailableLocales**() | Returns an array of all locales for which the getInstance methods of this class can return localized instances. |
| String | **getCalendarType**() | Returns the calendar type of this Calendar. |
| String | **getDisplayName**(int field, int style, Locale locale) | Returns the string representation of the calendar field value in the given style and locale. |
| Map<String,Integer> | **getDisplayNames**(int field, int style, Locale locale) | Returns a Map containing all names of the calendar field in the given style and locale and their |

| | | |
|---|---|---|
| | | corresponding field values. |
| int | **getFirstDayOfWeek**() | Gets what the first day of the week is; e.g., SUNDAY in the U.S., MONDAY in France. |
| abstract int | **getGreatestMinimum**(int field) | Returns the highest minimum value for the given calendar field of this Calendar instance. |
| static Calendar | **getInstance**() | Gets a calendar using the default time zone and locale. |
| static Calendar | **getInstance**(Locale aLocale) | Gets a calendar using the default time zone and specified locale. |
| static Calendar | **getInstance**(TimeZone zone) | Gets a calendar using the specified time zone and default locale. |
| static Calendar | **getInstance**(TimeZone zone, Locale aLocale) | Gets a calendar with the specified time zone and locale. |
| abstract int | **getLeastMaximum**(int field) | Returns the lowest maximum value for the given calendar field of this Calendar instance. |
| abstract int | **getMaximum**(int field) | Returns the maximum value for the given calendar field of this Calendar instance. |
| int | **getMinimalDaysInFirstWee** | Gets what the minimal days required in the first |

| | | |
|---|---|---|
| | | week of the year are; e.g., if the first week is defined as one that contains the first day of the first month of a year, this method returns 1. |
| abstract int | **getMinimum**(int field) | Returns the minimum value for the given calendar field of this Calendar instance. |
| final Date | **getTime**() | Returns a Date object representing this Calendar's time value (millisecond offset from the Epoch"). |
| long | **getTimeInMillis**() | Returns this Calendar's time value in milliseconds. |
| TimeZone | **getTimeZone**() | Gets the time zone. |
| int | **getWeeksInWeekYear**() | Returns the number of weeks in the week year represented by this Calendar. |
| int | **getWeekYear**() | Returns the week year represented by this Calendar. |
| int | **hashCode**() | Returns a hash code for this calendar. |
| protected final int | **internalGet**(int field) | Returns the value of the given calendar field. |
| boolean | **isLenient**() | Tells whether date/time interpretation is to be lenient. |
| final boolean | **isSet**(int field) | Determines if the given calendar field has a |

| | | |
|---|---|---|
| | | value set, including cases that the value has been set by internal fields calculations triggered by a `get` method call. |
| `boolean` | **`isWeekDateSupported`**`()` | Returns whether this `Calendar` supports week dates. |
| `abstract void` | **`roll`**`(int field, boolean up)` | Adds or subtracts (up/down) a single unit of time on the given time field without changing larger fields. |
| `void` | **`roll`**`(int field, int amount)` | Adds the specified (signed) amount to the specified calendar field without changing larger fields. |
| `void` | **`set`**`(int field, int value)` | Sets the given calendar field to the given value. |
| `final void` | **`set`**`(int year, int month, int date)` | Sets the values for the calendar fields `YEAR`, `MONTH`, and `DAY_OF_MONTH`. |
| `final void` | **`set`**`(int year, int month, int date, int hourOfDay, int minute)` | Sets the values for the calendar fields `YEAR`, `MONTH`, `DAY_OF_MONTH`, `HOUR_OF_DAY`, and `MINUTE`. |
| `final void` | **`set`**`(int year, int month, int date, int hourOfDay, int minute, int second)` | Sets the values for the fields `YEAR`, `MONTH`, `DAY_OF_MONTH`, `HOUR_OF_DAY`, `MINUTE`, and `SECOND`. |
| `void` | | |

| | | |
|---|---|---|
| | **setFirstDayOfWeek**(int value) | Sets what the first day of the week is; e.g., SUNDAY in the U.S., MONDAY in France. |
| void | **setLenient**(boolean lenient) | Specifies whether or not date/time interpretation is to be lenient. |
| void | **setMinimalDaysInFirstWee**(int value) | Sets what the minimal days required in the first week of the year are; For example, if the first week is defined as one that contains the first day of the first month of a year, call this method with value 1. |
| final void | **setTime**(Date date) | Sets this Calendar's time with the given Date. |
| void | **setTimeInMillis**(long millis) | Sets this Calendar's current time from the given long value. |
| void | **setTimeZone**(TimeZone value) | Sets the time zone with the given time zone value. |
| void | **setWeekDate**(int weekYear, int weekOfYear, int dayOfWeek) | Sets the date of this Calendar with the given date specifiers - week year, week of year, and day of week. |
| final Instant | **toInstant**() | Converts this object to an Instant. |
| String | **toString**() | Return a string representation of this calendar. |

**Methods declared in class Object**

`finalize`, `getClass`, `notify`, `notifyAll`, `wait`, `wait`, `wait`

## *Field Details*

### ERA

```
public static final int ERA
```

Field number for `get` and `set` indicating the era, e.g., AD or BC in the Julian calendar. This is a calendar-specific value; see subclass documentation.

**See Also:**

`GregorianCalendar.AD`, `GregorianCalendar.BC`, Constant Field Values

### YEAR

```
public static final int YEAR
```

Field number for `get` and `set` indicating the year. This is a calendar-specific value; see subclass documentation.

**See Also:**

Constant Field Values

### MONTH

```
public static final int MONTH
```

Field number for `get` and `set` indicating the month. This is a calendar-specific value. The first month of the year in the Gregorian and Julian calendars is `JANUARY` which is 0; the last depends on the number of months in a year.

**See Also:**

`JANUARY`, `FEBRUARY`, `MARCH`, `APRIL`, `MAY`, `JUNE`, `JULY`, `AUGUST`, `SEPTEMBER`, `OCTOBER`, `NOVEMBER`, `DECEMBER`, `UNDECIMBER`, Constant Field Values

### WEEK_OF_YEAR

```
public static final int WEEK_OF_YEAR
```

Field number for `get` and `set` indicating the week number within the current year. The first week of the year, as defined by `getFirstDayOfWeek()` and `getMinimalDaysInFirstWeek()`, has value 1. Subclasses define the value of `WEEK_OF_YEAR` for days before the first week of the year.

**See Also:**

`getFirstDayOfWeek()`, `getMinimalDaysInFirstWeek()`, Constant Field Values

## WEEK_OF_MONTH

```
public static final int WEEK_OF_MONTH
```

Field number for `get` and `set` indicating the week number within the current month. The first week of the month, as defined by `getFirstDayOfWeek()` and `getMinimalDaysInFirstWeek()`, has value 1. Subclasses define the value of `WEEK_OF_MONTH` for days before the first week of the month.

**See Also:**

`getFirstDayOfWeek()`, `getMinimalDaysInFirstWeek()`, Constant Field Values

## DATE

```
public static final int DATE
```

Field number for `get` and `set` indicating the day of the month. This is a synonym for `DAY_OF_MONTH`. The first day of the month has value 1.

**See Also:**

`DAY_OF_MONTH`, Constant Field Values

## DAY_OF_MONTH

```
public static final int DAY_OF_MONTH
```

Field number for `get` and `set` indicating the day of the month. This is a synonym for `DATE`. The first day of the month has value 1.

**See Also:**

`DATE`, Constant Field Values

## DAY_OF_YEAR

```
public static final int DAY_OF_YEAR
```

Field number for `get` and `set` indicating the day number within the current year. The first day of the year has value 1.

**See Also:**

Constant Field Values

## DAY_OF_WEEK

```
public static final int DAY_OF_WEEK
```

Field number for `get` and `set` indicating the day of the week. If the calendar is non-lenient, this field takes values `SUNDAY`, `MONDAY`, `TUESDAY`, `WEDNESDAY`, `THURSDAY`, `FRIDAY`, and `SATURDAY`. Otherwise, any int values are accepted and normalized to one of the previously mentioned values.

**See Also:**

`SUNDAY`, `MONDAY`, `TUESDAY`, `WEDNESDAY`, `THURSDAY`, `FRIDAY`, `SATURDAY`, Constant Field Values

## DAY_OF_WEEK_IN_MONTH

```
public static final int DAY_OF_WEEK_IN_MONTH
```

Field number for `get` and `set` indicating the ordinal number of the day of the week within the current month. Together with the `DAY_OF_WEEK` field, this uniquely specifies a day within a month. Unlike `WEEK_OF_MONTH` and `WEEK_OF_YEAR`, this field's value does *not* depend on `getFirstDayOfWeek()` or `getMinimalDaysInFirstWeek()`. `DAY_OF_MONTH 1` through `7` always correspond to `DAY_OF_WEEK_IN_MONTH 1`; `8` through `14` correspond to `DAY_OF_WEEK_IN_MONTH 2`, and so on. `DAY_OF_WEEK_IN_MONTH 0` indicates the week before `DAY_OF_WEEK_IN_MONTH 1`. Negative values count back from the end of the month, so the last Sunday of a month is specified as `DAY_OF_WEEK = SUNDAY, DAY_OF_WEEK_IN_MONTH = -1`. Because negative values count backward they will usually be aligned differently within the month than positive values. For example, if a month has 31 days, `DAY_OF_WEEK_IN_MONTH -1` will overlap `DAY_OF_WEEK_IN_MONTH 5` and the end of `4`.

**See Also:**

`DAY_OF_WEEK`, `WEEK_OF_MONTH`, Constant Field Values

## AM_PM

```
public static final int AM_PM
```

Field number for `get` and `set` indicating whether the `HOUR` is before or after noon. E.g., at 10:04:15.250 PM the `AM_PM` is `PM`.

**See Also:**

`AM`, `PM`, `HOUR`, Constant Field Values

## HOUR

```
public static final int HOUR
```

Field number for `get` and `set` indicating the hour of the morning or afternoon. `HOUR` is used for the 12-hour clock (0 - 11). Noon and midnight are represented by 0, not by 12. E.g., at 10:04:15.250 PM the `HOUR` is 10.

**See Also:**

`AM_PM`, `HOUR_OF_DAY`, Constant Field Values

## HOUR_OF_DAY

```
public static final int HOUR_OF_DAY
```

Field number for `get` and `set` indicating the hour of the day. `HOUR_OF_DAY` is used for the 24-hour clock. E.g., at 10:04:15.250 PM the `HOUR_OF_DAY` is 22.

**See Also:**

`HOUR`, Constant Field Values

## MINUTE

```
public static final int MINUTE
```

Field number for `get` and `set` indicating the minute within the hour. E.g., at 10:04:15.250 PM the `MINUTE` is 4.

**See Also:**

Constant Field Values

## SECOND

```
public static final int SECOND
```

Field number for `get` and `set` indicating the second within the minute. E.g., at 10:04:15.250 PM the `SECOND` is 15.

**See Also:**

Constant Field Values

## MILLISECOND

```
public static final int MILLISECOND
```

Field number for `get` and `set` indicating the millisecond within the second. E.g., at 10:04:15.250 PM the `MILLISECOND` is 250.

**See Also:**

Constant Field Values

## ZONE_OFFSET

```
public static final int ZONE_OFFSET
```

Field number for `get` and `set` indicating the raw offset from GMT in milliseconds.

This field reflects the correct GMT offset value of the time zone of this `Calendar` if the `TimeZone` implementation subclass supports historical GMT offset changes.

**See Also:**

Constant Field Values

### DST_OFFSET

```
public static final int DST_OFFSET
```

Field number for `get` and `set` indicating the daylight saving offset in milliseconds.

This field reflects the correct daylight saving offset value of the time zone of this `Calendar` if the `TimeZone` implementation subclass supports historical Daylight Saving Time schedule changes.

**See Also:**

Constant Field Values

### FIELD_COUNT

```
public static final int FIELD_COUNT
```

The number of distinct fields recognized by `get` and `set`. Field numbers range from `0..FIELD_COUNT-1`.

**See Also:**

Constant Field Values

### SUNDAY

```
public static final int SUNDAY
```

Value of the `DAY_OF_WEEK` field indicating Sunday.

**See Also:**

Constant Field Values

### MONDAY

```
public static final int MONDAY
```

Value of the `DAY_OF_WEEK` field indicating Monday.

**See Also:**

Constant Field Values

### TUESDAY

`public static final int TUESDAY`

Value of the `DAY_OF_WEEK` field indicating Tuesday.

**See Also:**

Constant Field Values

### WEDNESDAY

`public static final int WEDNESDAY`

Value of the `DAY_OF_WEEK` field indicating Wednesday.

**See Also:**

Constant Field Values

### THURSDAY

`public static final int THURSDAY`

Value of the `DAY_OF_WEEK` field indicating Thursday.

**See Also:**

Constant Field Values

### FRIDAY

`public static final int FRIDAY`

Value of the `DAY_OF_WEEK` field indicating Friday.

**See Also:**

Constant Field Values

### SATURDAY

`public static final int SATURDAY`

Value of the `DAY_OF_WEEK` field indicating Saturday.

**See Also:**

Constant Field Values

## JANUARY

```
public static final int JANUARY
```

Value of the MONTH field indicating the first month of the year in the Gregorian and Julian calendars.

**See Also:**

Constant Field Values

## FEBRUARY

```
public static final int FEBRUARY
```

Value of the MONTH field indicating the second month of the year in the Gregorian and Julian calendars.

**See Also:**

Constant Field Values

## MARCH

```
public static final int MARCH
```

Value of the MONTH field indicating the third month of the year in the Gregorian and Julian calendars.

**See Also:**

Constant Field Values

## APRIL

```
public static final int APRIL
```

Value of the MONTH field indicating the fourth month of the year in the Gregorian and Julian calendars.

**See Also:**

Constant Field Values

## MAY

```
public static final int MAY
```

Value of the MONTH field indicating the fifth month of the year in the Gregorian and Julian calendars.

**See Also:**

Constant Field Values

## JUNE

```
public static final int JUNE
```

Value of the MONTH field indicating the sixth month of the year in the Gregorian and Julian calendars.

**See Also:**

Constant Field Values

## JULY

```
public static final int JULY
```

Value of the MONTH field indicating the seventh month of the year in the Gregorian and Julian calendars.

**See Also:**

Constant Field Values

## AUGUST

```
public static final int AUGUST
```

Value of the MONTH field indicating the eighth month of the year in the Gregorian and Julian calendars.

**See Also:**

Constant Field Values

## SEPTEMBER

```
public static final int SEPTEMBER
```

Value of the MONTH field indicating the ninth month of the year in the Gregorian and Julian calendars.

**See Also:**

Constant Field Values

## OCTOBER

```
public static final int OCTOBER
```

Value of the MONTH field indicating the tenth month of the year in the Gregorian and Julian calendars.

**See Also:**

Constant Field Values

## NOVEMBER

```
public static final int NOVEMBER
```

Value of the MONTH field indicating the eleventh month of the year in the Gregorian and Julian calendars.

**See Also:**

Constant Field Values

## DECEMBER

```
public static final int DECEMBER
```

Value of the MONTH field indicating the twelfth month of the year in the Gregorian and Julian calendars.

**See Also:**

Constant Field Values

## UNDECIMBER

```
public static final int UNDECIMBER
```

Value of the MONTH field indicating the thirteenth month of the year. Although GregorianCalendar does not use this value, lunar calendars do.

**See Also:**

Constant Field Values

## AM

```
public static final int AM
```

Value of the AM_PM field indicating the period of the day from midnight to just before noon.

**See Also:**

Constant Field Values

## PM

```
public static final int PM
```

Value of the AM_PM field indicating the period of the day from noon to just before midnight.

**See Also:**

Constant Field Values

## ALL_STYLES

```
public static final int ALL_STYLES
```

A style specifier for `getDisplayNames` indicating names in all styles, such as "January" and "Jan".

**Since:**

1.6

**See Also:**

`SHORT_FORMAT`, `LONG_FORMAT`, `SHORT_STANDALONE`, `LONG_STANDALONE`, `SHORT`, `LONG`, Constant Field Values

## SHORT

```
public static final int SHORT
```

A style specifier for `getDisplayName` and `getDisplayNames` equivalent to `SHORT_FORMAT`.

**Since:**

1.6

**See Also:**

`SHORT_STANDALONE`, `LONG`, Constant Field Values

## LONG

```
public static final int LONG
```

A style specifier for `getDisplayName` and `getDisplayNames` equivalent to `LONG_FORMAT`.

**Since:**

1.6

**See Also:**

`LONG_STANDALONE`, `SHORT`, Constant Field Values

## NARROW_FORMAT

```
public static final int NARROW_FORMAT
```

A style specifier for `getDisplayName` and `getDisplayNames` indicating a narrow name used for format. Narrow names are typically single character strings, such as "M" for Monday.

**Since:**

1.8

**See Also:**

NARROW_STANDALONE, SHORT_FORMAT, LONG_FORMAT, Constant Field Values

## NARROW_STANDALONE

```
public static final int NARROW_STANDALONE
```

A style specifier for getDisplayName and getDisplayNames indicating a narrow name independently. Narrow names are typically single character strings, such as "M" for Monday.

**Since:**

1.8

**See Also:**

NARROW_FORMAT, SHORT_STANDALONE, LONG_STANDALONE, Constant Field Values

## SHORT_FORMAT

```
public static final int SHORT_FORMAT
```

A style specifier for getDisplayName and getDisplayNames indicating a short name used for format.

**Since:**

1.8

**See Also:**

SHORT_STANDALONE, LONG_FORMAT, LONG_STANDALONE, Constant Field Values

## LONG_FORMAT

```
public static final int LONG_FORMAT
```

A style specifier for getDisplayName and getDisplayNames indicating a long name used for format.

**Since:**

1.8

**See Also:**

LONG_STANDALONE, SHORT_FORMAT, SHORT_STANDALONE, Constant Field Values

## SHORT_STANDALONE

```
public static final int SHORT_STANDALONE
```

A style specifier for `getDisplayName` and `getDisplayNames` indicating a short name used independently, such as a month abbreviation as calendar headers.

**Since:**

1.8

**See Also:**

`SHORT_FORMAT`, `LONG_FORMAT`, `LONG_STANDALONE`, Constant Field Values

## LONG_STANDALONE

```
public static final int LONG_STANDALONE
```

A style specifier for `getDisplayName` and `getDisplayNames` indicating a long name used independently, such as a month name as calendar headers.

**Since:**

1.8

**See Also:**

`LONG_FORMAT`, `SHORT_FORMAT`, `SHORT_STANDALONE`, Constant Field Values

## fields

```
protected int[] fields
```

The calendar field values for the currently set time for this calendar. This is an array of `FIELD_COUNT` integers, with index values `ERA` through `DST_OFFSET`.

## isSet

```
protected boolean[] isSet
```

The flags which tell if a specified calendar field for the calendar is set. A new object has no fields set. After the first call to a method which generates the fields, they all remain set after that. This is an array of `FIELD_COUNT` booleans, with index values `ERA` through `DST_OFFSET`.

## time

```
protected long time
```

The currently set time for this calendar, expressed in milliseconds after January 1, 1970, 0:00:00 GMT.

**See Also:**

`isTimeSet`

### isTimeSet

```
protected boolean isTimeSet
```

True if then the value of `time` is valid. The time is made invalid by a change to an item of `field[]`.

**See Also:**

`time`

### areFieldsSet

```
protected boolean areFieldsSet
```

True if `fields[]` are in sync with the currently set time. If false, then the next attempt to get the value of a field will force a recomputation of all fields from the current value of `time`.

## *Constructor Details*

### Calendar

```
protected Calendar()
```

Constructs a Calendar with the default time zone and the default `FORMAT` locale.

**See Also:**

`TimeZone.getDefault()`

### Calendar

```
protected Calendar(TimeZone zone,
                   Locale aLocale)
```

Constructs a calendar with the specified time zone and locale.

**Parameters:**

`zone` - the time zone to use

`aLocale` - the locale for the week data

## *Method Details*

### getInstance

```
public static Calendar getInstance()
```

Gets a calendar using the default time zone and locale. The `Calendar` returned is based on the current time in the default time zone with the default `FORMAT` locale.

If the locale contains the time zone with "tz" Unicode extension, that time zone is used instead.

**Returns:**

a Calendar.

## getInstance

```
public static Calendar getInstance(TimeZone zone)
```

Gets a calendar using the specified time zone and default locale. The `Calendar` returned is based on the current time in the given time zone with the default `FORMAT` locale.

**Parameters:**

`zone` - the time zone to use

**Returns:**

a Calendar.

**Throws:**

`NullPointerException` - if `zone` is `null`

## getInstance

```
public static Calendar getInstance(Locale aLocale)
```

Gets a calendar using the default time zone and specified locale. The `Calendar` returned is based on the current time in the default time zone with the given locale.

If the locale contains the time zone with "tz" Unicode extension, that time zone is used instead.

**Parameters:**

`aLocale` - the locale for the week data

**Returns:**

a Calendar.

**Throws:**

`NullPointerException` - if `aLocale` is `null`

## getInstance

```
public static Calendar getInstance(TimeZone zone,
                                   Locale aLocale)
```

Gets a calendar with the specified time zone and locale. The `Calendar` returned is based on the current time in the given time zone with the given locale.

**Parameters:**

`zone` - the time zone to use

`aLocale` - the locale for the week data

**Returns:**

a Calendar.

**Throws:**

`NullPointerException` - if `zone` or `aLocale` is `null`

## getAvailableLocales

```
public static Locale[] getAvailableLocales()
```

Returns an array of all locales for which the `getInstance` methods of this class can return localized instances. At a minimum, the returned array must contain a `Locale` instance equal to `Locale.ROOT` and a `Locale` instance equal to `Locale.US`.

**Returns:**

An array of locales for which localized `Calendar` instances are available.

## computeTime

```
protected abstract void computeTime()
```

Converts the current calendar field values in `fields[]` to the millisecond time value `time`.

**See Also:**

`complete()`, `computeFields()`

## computeFields

```
protected abstract void computeFields()
```

Converts the current millisecond time value `time` to calendar field values in `fields[]`. This allows you to sync up the calendar field values with a new time that is set for the calendar. The time is *not* recomputed first; to recompute the time, then the fields, call the `complete()` method.

**See Also:**

computeTime()

## getTime

```
public final Date getTime()
```

Returns a Date object representing this Calendar's time value (millisecond offset from the Epoch").

**Returns:**

a Date representing the time value.

**See Also:**

setTime(Date), getTimeInMillis()

## setTime

```
public final void setTime(Date date)
```

Sets this Calendar's time with the given Date.

Note: Calling setTime() with Date(Long.MAX_VALUE) or Date(Long.MIN_VALUE) may yield incorrect field values from get().

**Parameters:**

date - the given Date.

**Throws:**

NullPointerException - if date is null

**See Also:**

getTime(), setTimeInMillis(long)

## getTimeInMillis

```
public long getTimeInMillis()
```

Returns this Calendar's time value in milliseconds.

**Returns:**

the current time as UTC milliseconds from the epoch.

**See Also:**

getTime(), setTimeInMillis(long)

## setTimeInMillis

```
public void setTimeInMillis(long millis)
```

Sets this Calendar's current time from the given long value.

**Parameters:**

`millis` - the new time in UTC milliseconds from the epoch.

**See Also:**

`setTime(Date)`, `getTimeInMillis()`

## get

```
public int get(int field)
```

Returns the value of the given calendar field. In lenient mode, all calendar fields are normalized. In non-lenient mode, all calendar fields are validated and this method throws an exception if any calendar fields have out-of-range values. The normalization and validation are handled by the `complete()` method, which process is calendar system dependent.

**Parameters:**

`field` - the given calendar field.

**Returns:**

the value for the given calendar field.

**Throws:**

`IllegalArgumentException` - if this `Calendar` is non-lenient and any of the calendar fields have invalid values.

**See Also:**

`set(int,int)`,
`complete()`

## internalGet

```
protected final int internalGet(int field)
```

Returns the value of the given calendar field. This method does not involve normalization or validation of the field value.

**Parameters:**

`field` - the given calendar field.

**Returns:**

the value for the given calendar field.

**See Also:**

`get(int)`

## set

```
public void set(int field,
                int value)
```

Sets the given calendar field to the given value. The value is not interpreted by this method regardless of the leniency mode.

**Parameters:**

`field` - the given calendar field.

`value` - the value to be set for the given calendar field.

**See Also:**

`set(int,int,int)`,
`set(int,int,int,int,int)`,
`set(int,int,int,int,int,int)`,
`get(int)`

## set

```
public final void set(int year,
                      int month,
                      int date)
```

Sets the values for the calendar fields `YEAR`, `MONTH`, and `DAY_OF_MONTH`. Previous
values of other calendar fields are retained. If this is not desired, call `clear()` first.

**Parameters:**

`year` - the value used to set the `YEAR` calendar field.

`month` - the value used to set the `MONTH` calendar field. Month value is 0-based. e.g., 0 for January.

`date` - the value used to set the `DAY_OF_MONTH` calendar field.

**See Also:**

`set(int,int)`,
`set(int,int,int,int,int)`,
`set(int,int,int,int,int,int)`

## set

```
public final void set(int year,
                      int month,
                      int date,
                      int hourOfDay,
                      int minute)
```

Sets the values for the calendar fields YEAR, MONTH, DAY_OF_MONTH, HOUR_OF_DAY, and MINUTE. Previous values of other fields are retained. If this is not desired, call clear() first.

**Parameters:**

year - the value used to set the YEAR calendar field.

month - the value used to set the MONTH calendar field. Month value is 0-based. e.g., 0 for January.

date - the value used to set the DAY_OF_MONTH calendar field.

hourOfDay - the value used to set the HOUR_OF_DAY calendar field.

minute - the value used to set the MINUTE calendar field.

**See Also:**

set(int,int),
set(int,int,int),
set(int,int,int,int,int)

## set

```
public final void set(int year,
                      int month,
                      int date,
                      int hourOfDay,
                      int minute,
                      int second)
```

Sets the values for the fields YEAR, MONTH, DAY_OF_MONTH, HOUR_OF_DAY, MINUTE, and SECOND. Previous values of other fields are retained. If this is not desired, call clear() first.

**Parameters:**

year - the value used to set the YEAR calendar field.

month - the value used to set the MONTH calendar field. Month value is 0-based. e.g., 0 for January.

date - the value used to set the DAY_OF_MONTH calendar field.

hourOfDay - the value used to set the HOUR_OF_DAY calendar field.

minute - the value used to set the MINUTE calendar field.

second - the value used to set the SECOND calendar field.

**See Also:**

set(int,int),
set(int,int,int),
set(int,int,int,int,int)

## clear

```
public final void clear()
```

Sets all the calendar field values and the time value (millisecond offset from the Epoch) of this `Calendar` undefined. This means that `isSet()` will return `false` for all the calendar fields, and the date and time calculations will treat the fields as if they had never been set. A `Calendar` implementation class may use its specific default field values for date/time calculations. For example, `GregorianCalendar` uses 1970 if the `YEAR` field value is undefined.

**See Also:**

`clear(int)`

## clear

```
public final void clear(int field)
```

Sets the given calendar field value and the time value (millisecond offset from the Epoch) of this `Calendar` undefined. This means that `isSet(field)` will return `false`, and the date and time calculations will treat the field as if it had never been set. A `Calendar` implementation class may use the field's specific default value for date and time calculations.

The `HOUR_OF_DAY`, `HOUR` and `AM_PM` fields are handled independently and the the resolution rule for the time of day is applied. Clearing one of the fields doesn't reset the hour of day value of this `Calendar`. Use `set(Calendar.HOUR_OF_DAY, 0)` to reset the hour value.

**Parameters:**

`field` - the calendar field to be cleared.

**See Also:**

`clear()`

## isSet

```
public final boolean isSet(int field)
```

Determines if the given calendar field has a value set, including cases that the value has been set by internal fields calculations triggered by a `get` method call.

**Parameters:**

`field` - the calendar field to test

**Returns:**

`true` if the given calendar field has a value set; `false` otherwise.

### getDisplayName

```
public String getDisplayName(int field,
                             int style,
                             Locale locale)
```

Returns the string representation of the calendar `field` value in the given `style` and `locale`. If no string representation is applicable, `null` is returned. This method calls `get(field)` to get the calendar `field` value if the string representation is applicable to the given calendar `field`.

For example, if this `Calendar` is a `GregorianCalendar` and its date is 2005-01-01, then the string representation of the `MONTH` field would be "January" in the long style in an English locale or "Jan" in the short style. However, no string representation would be available for the `DAY_OF_MONTH` field, and this method would return `null`.

The default implementation supports the calendar fields for which a `DateFormatSymbols` has names in the given `locale`.

If there is no string representation of the `Calendar` `field` and the calendar is in non-lenient mode and any calendar fields have invalid values, `null` is returned. If there is a string representation of the `Calendar` `field` and the calendar is in non-lenient mode and any calendar fields have invalid values, `IllegalArgumentException` will be thrown.

**Parameters:**

`field` - the calendar field for which the string representation is returned

`style` - the style applied to the string representation; one of `SHORT_FORMAT` (`SHORT`), `SHORT_STANDALONE`, `LONG_FORMAT` (`LONG`), `LONG_STANDALONE`, `NARROW_FORMAT`, or `NARROW_STANDALONE`.

`locale` - the locale for the string representation (any calendar types specified by `locale` are ignored)

**Returns:**

the string representation of the given `field` in the given `style`, or `null` if no string representation is applicable.

**Throws:**

`IllegalArgumentException` - if `field` or `style` is invalid, or if this `Calendar` is non-lenient and any of the calendar fields have invalid values

`NullPointerException` - if `locale` is null

**Since:**

1.6

### getDisplayNames

```
public Map<String,Integer> getDisplayNames(int field,
                                           int style,
                                           Locale locale)
```

Returns a `Map` containing all names of the calendar `field` in the given `style` and `locale` and their corresponding field values. For example, if this `Calendar` is a `GregorianCalendar`, the returned map would contain "Jan" to `JANUARY`, "Feb" to `FEBRUARY`, and so on, in the `short` style in an English locale.

Narrow names may not be unique due to use of single characters, such as "S" for Sunday and Saturday. In that case narrow names are not included in the returned `Map`.

The values of other calendar fields may be taken into account to determine a set of display names. For example, if this `Calendar` is a lunisolar calendar system and the year value given by the `YEAR` field has a leap month, this method would return month names containing the leap month name, and month names are mapped to their values specific for the year.

The default implementation supports display names contained in a `DateFormatSymbols`. For example, if `field` is `MONTH` and `style` is `ALL_STYLES`, this method returns a `Map` containing all strings returned by `DateFormatSymbols.getShortMonths()` and `DateFormatSymbols.getMonths()`.

Unlike `getDisplayName(int, int, Locale)`, this method will not throw an `IllegalArgumentException` if the `Calendar` is non-lenient and any of the calendar fields have invalid values. Instead, this method will return either `null` or a `Map`.

**Parameters:**

`field` - the calendar field for which the display names are returned

`style` - the style applied to the string representation; one of `SHORT_FORMAT` (`SHORT`), `SHORT_STANDALONE`, `LONG_FORMAT` (`LONG`), `LONG_STANDALONE`, `NARROW_FORMAT`, or `NARROW_STANDALONE`

`locale` - the locale for the display names

**Returns:**

a `Map` containing all display names in `style` and `locale` and their field values, or `null` if no display names are defined for `field`

**Throws:**

`IllegalArgumentException` - if `field` or `style` is invalid

`NullPointerException` - if `locale` is null

**Since:**

1.6

## complete

```
protected void complete()
```

Fills in any unset fields in the calendar fields. First, the `computeTime()` method is called if the time value (millisecond offset from the Epoch) has not been calculated from calendar field values. Then, the `computeFields()` method is called to calculate all calendar field values.

## getAvailableCalendarTypes

```
public static Set<String> getAvailableCalendarTypes()
```

Returns an unmodifiable `Set` containing all calendar types supported by `Calendar` in the runtime environment. The available calendar types can be used for the Unicode locale extensions. The `Set` returned contains at least `"gregory"`. The calendar types don't include aliases, such as `"gregorian"` for `"gregory"`.

**Returns:**

an unmodifiable `Set` containing all available calendar types

**Since:**

1.8

**See Also:**

`getCalendarType()`,
`Calendar.Builder.setCalendarType(String)`,
`Locale.getUnicodeLocaleType(String)`

## getCalendarType

```
public String getCalendarType()
```

Returns the calendar type of this `Calendar`. Calendar types are defined by the *Unicode Locale Data Markup Language (LDML)* specification.

The default implementation of this method returns the class name of this `Calendar` instance. Any subclasses that implement LDML-defined calendar systems should override this method to return appropriate calendar types.

**Returns:**

the LDML-defined calendar type or the class name of this `Calendar` instance

**Since:**

1.8

**See Also:**

Locale extensions,
`Locale.Builder.setLocale(Locale)`,
`Locale.Builder.setUnicodeLocaleKeyword(String, String)`

## equals

```
public boolean equals(Object obj)
```

Compares this `Calendar` to the specified `Object`. The result is `true` if and only if the argument is a `Calendar` object of the same calendar system that represents the same time value (millisecond offset from the Epoch) under the same `Calendar` parameters as this object.

The `Calendar` parameters are the values represented by the `isLenient`, `getFirstDayOfWeek`, `getMinimalDaysInFirstWeek` and `getTimeZone` methods. If there is any difference in those parameters between the two `Calendars`, this method returns `false`.

Use the `compareTo` method to compare only the time values.

**Overrides:**

`equals` in class `Object`

**Parameters:**

`obj` - the object to compare with.

**Returns:**

`true` if this object is equal to `obj`; `false` otherwise.

**See Also:**

`Object.hashCode()`, `HashMap`

## hashCode

```
public int hashCode()
```

Returns a hash code for this calendar.

**Overrides:**

`hashCode` in class `Object`

**Returns:**

a hash code value for this object.

**Since:**

1.2

**See Also:**

`Object.equals(java.lang.Object)`,

System.identityHashCode(Object)

## before

public boolean before(Object when)

Returns whether this Calendar represents a time before the time represented by the specified Object. This method is equivalent to:

```
        compareTo(when) < 0
```

if and only if when is a Calendar instance. Otherwise, the method returns false.

**Parameters:**

when - the Object to be compared

**Returns:**

true if the time of this Calendar is before the time represented by when; false otherwise.

**See Also:**

compareTo(Calendar)

## after

public boolean after(Object when)

Returns whether this Calendar represents a time after the time represented by the specified Object. This method is equivalent to:

```
        compareTo(when) > 0
```

if and only if when is a Calendar instance. Otherwise, the method returns false.

**Parameters:**

when - the Object to be compared

**Returns:**

true if the time of this Calendar is after the time represented by when; false otherwise.

**See Also:**

compareTo(Calendar)

## compareTo

```
public int compareTo(Calendar anotherCalendar)
```

Compares the time values (millisecond offsets from the Epoch) represented by two `Calendar` objects.

**Specified by:**

`compareTo` in interface `Comparable<Calendar>`

**Parameters:**

`anotherCalendar` - the `Calendar` to be compared.

**Returns:**

the value `0` if the time represented by the argument is equal to the time represented by this `Calendar`; a value less than `0` if the time of this `Calendar` is before the time represented by the argument; and a value greater than `0` if the time of this `Calendar` is after the time represented by the argument.

**Throws:**

`NullPointerException` - if the specified `Calendar` is `null`.

`IllegalArgumentException` - if the time value of the specified `Calendar` object can't be obtained due to any invalid calendar values.

**Since:**

1.5

## add

```
public abstract void add(int field,
                         int amount)
```

Adds or subtracts the specified amount of time to the given calendar field, based on the calendar's rules. For example, to subtract 5 days from the current time of the calendar, you can achieve it by calling:

`add(Calendar.DAY_OF_MONTH, -5)`.

**Parameters:**

`field` - the calendar field.

`amount` - the amount of date or time to be added to the field.

**Throws:**

`IllegalArgumentException` - if this `Calendar` is non-lenient and any of the calendar fields have invalid values or if `field` is `ZONE_OFFSET`, `DST_OFFSET`, or unknown.

**See Also:**

`roll(int,int)`,
`set(int,int)`

## roll

```
public abstract void roll(int field,
                          boolean up)
```

Adds or subtracts (up/down) a single unit of time on the given time field without changing larger fields. For example, to roll the current date up by one day, you can achieve it by calling:

roll(Calendar.DATE, true). When rolling on the year or Calendar.YEAR field, it will roll the year value in the range between 1 and the value returned by calling `getMaximum(Calendar.YEAR)`. When rolling on the month or Calendar.MONTH field, other fields like date might conflict and, need to be changed. For instance, rolling the month on the date 01/31/96 will result in 02/29/96. When rolling on the hour-in-day or Calendar.HOUR_OF_DAY field, it will roll the hour value in the range between 0 and 23, which is zero-based.

**Parameters:**

`field` - the time field.

`up` - indicates if the value of the specified time field is to be rolled up or rolled down. Use true if rolling up, false otherwise.

**Throws:**

`IllegalArgumentException` - if this `Calendar` is non-lenient and any of the calendar fields have invalid values or if `field` is `ZONE_OFFSET`, `DST_OFFSET`, or unknown.

**See Also:**

`add(int,int)`,
`set(int,int)`

## roll

```
public void roll(int field,
                 int amount)
```

Adds the specified (signed) amount to the specified calendar field without changing larger fields. A negative amount means to roll down.

NOTE: This default implementation on `Calendar` just repeatedly calls the version of `roll()` that rolls by one unit. This may not always do the right thing. For example, if the `DAY_OF_MONTH` field is 31, rolling through February will leave it set to 28. The `GregorianCalendar` version of this function takes

care of this problem. Other subclasses should also provide overrides of this function that do the right thing.

**Parameters:**

`field` - the calendar field.

`amount` - the signed amount to add to the calendar `field`.

**Throws:**

`IllegalArgumentException` - if this `Calendar` is non-lenient and any of the calendar fields have invalid values or if `field` is `ZONE_OFFSET`, `DST_OFFSET`, or unknown.

**Since:**

1.2

**See Also:**

`roll(int,boolean)`,
`add(int,int)`,
`set(int,int)`

## setTimeZone

```
public void setTimeZone(TimeZone value)
```

Sets the time zone with the given time zone value.

**Parameters:**

`value` - the given time zone.

## getTimeZone

```
public TimeZone getTimeZone()
```

Gets the time zone.

**Returns:**

the time zone object associated with this calendar.

## setLenient

```
public void setLenient(boolean lenient)
```

Specifies whether or not date/time interpretation is to be lenient. With lenient interpretation, a date such as "February 942, 1996" will be treated as being equivalent to the 941st day after February 1, 1996. With strict (non-lenient) interpretation, such dates will cause an exception to be thrown. The default is lenient.

**Parameters:**

`lenient` - `true` if the lenient mode is to be turned on; `false` if it is to be turned off.

**See Also:**

`isLenient()`, `DateFormat.setLenient(boolean)`

## isLenient

```
public boolean isLenient()
```

Tells whether date/time interpretation is to be lenient.

**Returns:**

`true` if the interpretation mode of this calendar is lenient; `false` otherwise.

**See Also:**

`setLenient(boolean)`

## setFirstDayOfWeek

```
public void setFirstDayOfWeek(int value)
```

Sets what the first day of the week is; e.g., `SUNDAY` in the U.S., `MONDAY` in France.

**Parameters:**

`value` - the given first day of the week.

**See Also:**

`getFirstDayOfWeek()`, `getMinimalDaysInFirstWeek()`

## getFirstDayOfWeek

```
public int getFirstDayOfWeek()
```

Gets what the first day of the week is; e.g., `SUNDAY` in the U.S., `MONDAY` in France.

**Returns:**

the first day of the week.

**See Also:**

`setFirstDayOfWeek(int)`, `getMinimalDaysInFirstWeek()`

## setMinimalDaysInFirstWeek

```
public void setMinimalDaysInFirstWeek(int value)
```

Sets what the minimal days required in the first week of the year are; For example, if the first week is defined as one that contains the first day of the

first month of a year, call this method with value 1. If it must be a full week, use value 7.

**Parameters:**

`value` - the given minimal days required in the first week of the year.

**See Also:**

`getMinimalDaysInFirstWeek()`

## getMinimalDaysInFirstWeek

```
public int getMinimalDaysInFirstWeek()
```

Gets what the minimal days required in the first week of the year are; e.g., if the first week is defined as one that contains the first day of the first month of a year, this method returns 1. If the minimal days required must be a full week, this method returns 7.

**Returns:**

the minimal days required in the first week of the year.

**See Also:**

`setMinimalDaysInFirstWeek(int)`

## isWeekDateSupported

```
public boolean isWeekDateSupported()
```

Returns whether this `Calendar` supports week dates.

The default implementation of this method returns `false`.

**Returns:**

`true` if this `Calendar` supports week dates; `false` otherwise.

**Since:**

1.7

**See Also:**

`getWeekYear()`,
`setWeekDate(int,int,int)`,
`getWeeksInWeekYear()`

## getWeekYear

```
public int getWeekYear()
```

Returns the week year represented by this `Calendar`. The week year is in sync with the week cycle. The first day of the first week is the first day of the week year.

The default implementation of this method throws an `UnsupportedOperationException`.

**Returns:**

the week year of this `Calendar`

**Throws:**

`UnsupportedOperationException` - if any week year numbering isn't supported in this `Calendar`.

**Since:**

1.7

**See Also:**

`isWeekDateSupported()`, `getFirstDayOfWeek()`, `getMinimalDaysInFirstWeek()`

## setWeekDate

```
public void setWeekDate(int weekYear,
                        int weekOfYear,
                        int dayOfWeek)
```

Sets the date of this `Calendar` with the given date specifiers - week year, week of year, and day of week.

Unlike the `set` method, all of the calendar fields and `time` values are calculated upon return.

If `weekOfYear` is out of the valid week-of-year range in `weekYear`, the `weekYear` and `weekOfYear` values are adjusted in lenient mode, or an `IllegalArgumentException` is thrown in non-lenient mode.

The default implementation of this method throws an `UnsupportedOperationException`.

**Parameters:**

`weekYear` - the week year

`weekOfYear` - the week number based on `weekYear`

`dayOfWeek` - the day of week value: one of the constants for the `DAY_OF_WEEK` field: `SUNDAY`, ..., `SATURDAY`.

**Throws:**

`IllegalArgumentException` - if any of the given date specifiers is invalid or any of the calendar fields are inconsistent with the given date specifiers in non-lenient mode

UnsupportedOperationException - if any week year numbering isn't supported in this Calendar.

**Since:**

1.7

**See Also:**

isWeekDateSupported(), getFirstDayOfWeek(), getMinimalDaysInFirstWeek()

## getWeeksInWeekYear

```
public int getWeeksInWeekYear()
```

Returns the number of weeks in the week year represented by this Calendar.

The default implementation of this method throws an UnsupportedOperationException.

**Returns:**

the number of weeks in the week year.

**Throws:**

UnsupportedOperationException - if any week year numbering isn't supported in this Calendar.

**Since:**

1.7

**See Also:**

WEEK_OF_YEAR, isWeekDateSupported(), getWeekYear(), getActualMaximum(int)

## getMinimum

```
public abstract int getMinimum(int field)
```

Returns the minimum value for the given calendar field of this Calendar instance. The minimum value is defined as the smallest value returned by the get method for any possible time value. The minimum value depends on calendar system specific parameters of the instance.

**Parameters:**

field - the calendar field.

**Returns:**

the minimum value for the given calendar field.

**See Also:**

`getMaximum(int)`, `getGreatestMinimum(int)`, `getLeastMaximum(int)`, `getActualMinimum(int)`, `getActualMaximum(int)`

## getMaximum

```
public abstract int getMaximum(int field)
```

Returns the maximum value for the given calendar field of this `Calendar` instance. The maximum value is defined as the largest value returned by the `get` method for any possible time value. The maximum value depends on calendar system specific parameters of the instance.

**Parameters:**

`field` - the calendar field.

**Returns:**

the maximum value for the given calendar field.

**See Also:**

`getMinimum(int)`, `getGreatestMinimum(int)`, `getLeastMaximum(int)`, `getActualMinimum(int)`, `getActualMaximum(int)`

## getGreatestMinimum

```
public abstract int getGreatestMinimum(int field)
```

Returns the highest minimum value for the given calendar field of this `Calendar` instance. The highest minimum value is defined as the largest value returned by `getActualMinimum(int)` for any possible time value. The greatest minimum value depends on calendar system specific parameters of the instance.

**Parameters:**

`field` - the calendar field.

**Returns:**

the highest minimum value for the given calendar field.

**See Also:**

`getMinimum(int)`, `getMaximum(int)`, `getLeastMaximum(int)`, `getActualMinimum(int)`, `getActualMaximum(int)`

## getLeastMaximum

```
public abstract int getLeastMaximum(int field)
```

Returns the lowest maximum value for the given calendar field of this `Calendar` instance. The lowest maximum value is defined as the smallest value

returned by `getActualMaximum(int)` for any possible time value. The least maximum value depends on calendar system specific parameters of the instance. For example, a `Calendar` for the Gregorian calendar system returns 28 for the `DAY_OF_MONTH` field, because the 28th is the last day of the shortest month of this calendar, February in a common year.

**Parameters:**

`field` - the calendar field.

**Returns:**

the lowest maximum value for the given calendar field.

**See Also:**

`getMinimum(int)`, `getMaximum(int)`, `getGreatestMinimum(int)`, `getActualMinimum(int)`, `getActualMaximum(int)`

## getActualMinimum

```
public int getActualMinimum(int field)
```

Returns the minimum value that the specified calendar field could have, given the time value of this `Calendar`.

The default implementation of this method uses an iterative algorithm to determine the actual minimum value for the calendar field. Subclasses should, if possible, override this with a more efficient implementation - in many cases, they can simply return `getMinimum()`.

**Parameters:**

`field` - the calendar field

**Returns:**

the minimum of the given calendar field for the time value of this `Calendar`

**Since:**

1.2

**See Also:**

`getMinimum(int)`, `getMaximum(int)`, `getGreatestMinimum(int)`, `getLeastMaximum(int)`, `getActualMaximum(int)`

## getActualMaximum

```
public int getActualMaximum(int field)
```

Returns the maximum value that the specified calendar field could have, given the time value of this `Calendar`.
For example, the actual maximum value of the `MONTH` field is 12 in some years, and 13 in other years in the Hebrew calendar system.

The default implementation of this method uses an iterative algorithm to determine the actual maximum value for the calendar field. Subclasses should, if possible, override this with a more efficient implementation.

**Parameters:**

`field` - the calendar field

**Returns:**

the maximum of the given calendar field for the time value of this `Calendar`

**Since:**

1.2

**See Also:**

`getMinimum(int)`, `getMaximum(int)`, `getGreatestMinimum(int)`, `getLeastMaximum(int)`, `getActualMinimum(int)`

## clone

```
public Object clone()
```

Creates and returns a copy of this object.

**Overrides:**

`clone` in class `Object`

**Returns:**

a copy of this object.

**See Also:**

`Cloneable`

## toString

```
public String toString()
```

Return a string representation of this calendar. This method is intended to be used only for debugging purposes, and the format of the returned string may vary between implementations. The returned string may be empty but may not be `null`.

**Overrides:**

`toString` in class `Object`

**Returns:**

a string representation of this calendar.

## toInstant

```
public final Instant toInstant()
```

Converts this object to an `Instant`.

The conversion creates an `Instant` that represents the same point on the time-line as this `Calendar`.

**Returns:**

the instant representing the same point on the time-line

**Since:**

1.8

---

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.